Mini Assignment: SSA, Dominators and Post Dominators

Question 5: LLVM mem2reg Experiment

CS3423 – Compilers II

November 2026

1 Objective

The objective of this experiment is to understand how LLVM's `mem2reg` pass promotes eligible stack-allocated variables into Static Single Assignment (SSA) form. Five C++ programs involving pointers, conditional branches, function calls, arrays, loops and dynamic memory allocation were compiled into LLVM IR. The IR before and after applying `mem2reg` was compared to identify which variables were promoted and which remained in memory.

2 Experimental Setup

- Operating environment: Ubuntu on WSL2
- Clang version: 18.1.3
- LLVM version: 18.1.3
- Language standard: C++17
- Optimization level: O0

The initial LLVM IR was generated using:

```
clang++ -std=c++17 -S -emit-llvm -O0 \
    -Xclang -disable-O0-optnone source.cpp -o before.ll
```

The `-disable-O0-optnone` option prevents Clang from attaching the `optnone` attribute, which would otherwise prevent the optimization pass from running.

The transformation was performed using:

```
opt -passes=mem2reg -S before.ll -o after.ll
```

The transformed IR was checked using:

```
opt -passes=verify -disable-output after.ll
```

All five transformed modules passed LLVM verification.

3 Analysis Information Used by mem2reg

The LLVM 18.1.3 pass-manager diagnostics were collected using:

```
opt -debug-pass-manager -passes=mem2reg \
    -disable-output before.ll
```

The diagnostics reported the following analyses:

- **DominatorTreeAnalysis:** Provides the dominator tree of a function. Dominance information is important for SSA construction and renaming of variable definitions.
- **AssumptionAnalysis:** Provides access to the function's assumption cache.

- **TargetIRAnalysis:** Provides target-specific information about the function's IR.
- **VerifierAnalysis:** Used by the verifier to validate the transformed module.

The pass-manager output also reported `InnerAnalysisManagerProxy`, which is infrastructure connecting the module and function analysis managers.

For SSA construction, dominance frontiers identify locations where definitions from different control-flow paths meet and where PHI nodes may be required. The dominator tree is also used during the renaming process.

The diagnostics establish which analyses were requested by the pass manager; they do not imply that every reported analysis directly participates in PHI-node insertion.

4 Experimental Programs and Observations

4.1 Program 1: Basic Pointer Indirection

The program declares an integer variable and a pointer to it. The pointer is used to update the integer.

Observed result: All three stack allocations disappeared after promotion. The loads and stores associated with them also disappeared.

The resulting function was simplified to:

```
%2 = add nsw i32 10, %0
ret i32 %2
```

Explanation: LLVM was able to promote both the integer and pointer allocations. Although the integer was accessed indirectly through a pointer, the pointer's uses were sufficiently restricted for LLVM's promotion checks to establish that the allocation was safe to promote.

4.2 Program 2: Conditional Pointer Reassignment

Two integer variables are declared. Depending on a condition, a pointer is assigned the address of one of them.

The transformed IR contains:

```
%.0 = phi ptr [ %2, %5 ], [ %3, %6 ]
%8 = load i32, ptr %.0
%9 = add nsw i32 %8, 50
store i32 %9, ptr %.0
```

Observed result: The pointer variable's stack allocation was eliminated, but the two integer allocations remained.

Explanation: The pointer itself was promoted into SSA form. A pointer-valued PHI node selects the appropriate address depending on the predecessor block. The integers remain in memory because their addresses are accessed indirectly through the selected pointer.

This demonstrates that a pointer can be represented in SSA form even when the memory it references is not promoted.

4.3 Program 3: Pointer Passed to Another Function

The program passes a pointer to a local integer into a separate function that modifies the pointed-to value.

Observed result: The pointer variable's stack allocation was promoted, but the integer's allocation remained.

The transformed function contains:

```
call void @_Z6modifyPi(ptr noundef %2)
%3 = load i32, ptr %2, align 4
ret i32 %3
```

Explanation: The integer's address is passed to another function. That function accesses and modifies the memory through its pointer argument. Consequently, the integer remains memory-backed. The pointer variable's own stack slot can still be eliminated.

The pointer argument of the `modify` function is already an SSA function argument; its being an SSA value does not mean that the pointed-to memory is in SSA form.

4.4 Program 4: Array Pointer Arithmetic and Loops

The program traverses an array using pointer arithmetic and accumulates its elements in a loop.

The transformed IR contains the following PHI nodes:

```
%.01 = phi i32 [ 0, %1 ], [ %14, %15 ]
%.0 = phi i32 [ 0, %1 ], [ %16, %15 ]
%9 = phi i1 [ false, %4 ], [ %7, %6 ]
```

Observed result: The scalar variables representing the accumulator, loop counter and pointer variable were promoted. The array allocation remained.

Explanation: The accumulator and loop counter receive new values on every iteration, so PHI nodes represent their values at the loop header. Another PHI node combines the Boolean results of the loop conditions. The array remains memory-backed because its elements are accessed through computed addresses.

4.5 Program 5: Dynamically Allocated Memory

This program allocates an integer array dynamically using `new[]`, writes values into it and calculates their sum.

The transformed IR contains:

```
%.01 = phi i32 [ 0, %1 ], [ %18, %19 ]
%.0 = phi i32 [ 0, %1 ], [ %20, %19 ]

store i32 %12, ptr %14, align 4
%17 = load i32, ptr %16, align 4
```

Observed result: All four stack allocations disappeared. The loop counter and accumulator were represented using SSA values and PHI nodes. However, the heap allocation and its associated memory accesses remained.

Explanation: The pointer variable's own stack slot was promoted, but the dynamically allocated array is heap memory, not a stack allocation created by `alloca`. Therefore, its loads and stores remain.

5 Quantitative Comparison

The following counts were obtained by counting LLVM `alloca`, `load`, `store` and `phi` instructions in the complete modules before and after applying `mem2reg`.

	Before			After			After
Program	Alloca	Load	Store	Alloca	Load	Store	PHI
1	3	5	4	0	0	0	0
2	4	5	6	2	3	3	1
3	4	6	5	1	2	2	0
4	5	9	6	1	1	0	3
5	4	13	7	0	1	1	2

These counts are module-wide, including all functions such as `main` and helper functions. They are not counts restricted to the main experimental function.

6 Summary of Variable Promotion

Program	Variable	Observation
1	`value`	Promoted; all associated memory operations disappear.
1	`ptr`	Promoted; its stack allocation disappears.
2	`ptr`	Promoted; represented by a pointer PHI node.
2	`first`, `second`	Not promoted; accessed indirectly through the pointer.
3	`ptr`	Promoted; its stack allocation disappears.
3	`value`	Not promoted; its address is passed to another function.
3	`p`	Already an SSA function argument; accesses the pointee through memory.
4	`sum`, `i`	Promoted; PHI nodes represent loop-carried values.
4	`ptr`	Promoted; its stack allocation disappears.
4	`arr`	Not promoted; array memory remains.
5	`buffer`	Promoted; its pointer stack slot disappears.
5	`sum`, `i`	Promoted; loop-carried values are represented in SSA form.
5	Heap array	Not promoted; dynamically allocated memory remains.

Function parameters such as `input`, `condition` and `n` are already SSA values in LLVM IR. They are not variables newly promoted by `mem2reg`.

7 Conclusion

The experiments demonstrate that LLVM's `mem2reg` pass selectively promotes eligible stack allocations rather than converting every local variable into an SSA value.

Simple scalar variables and even pointer variables can be promoted when their stack allocations have suitable uses. PHI nodes represent values that depend on control-flow paths and loop iterations. However, variables accessed indirectly through pointers, addresses passed to other functions, arrays and dynamically allocated heap memory can remain memory-backed.

An important observation is that promoting a pointer does not necessarily promote the memory to which it points. SSA form applies to the pointer value independently of the pointee's memory representation.

Overall, `mem2reg` removes unnecessary stack memory operations and creates an SSA representation that makes subsequent compiler optimizations easier.